

1. PROCEDIMIENTOS

Un procedimiento es una especie de "función" que ejecuta varias instrucciones.

Sirve para agrupar código y reutilizarlo.

Ejemplo:

```
CREATE OR REPLACE PROCEDURE saludar
IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hola');
END;
/
```

Cuando lo llamas:

```
BEGIN
  saludar;
END;
/
```

Se ejecuta.

Estructura

Siempre tiene esta forma:

```
CREATE OR REPLACE PROCEDURE nombre
(parametros)
IS
  variables
BEGIN
  instrucciones
EXCEPTION
  errores
END;
/
```

Qué significa cada parte

CREATE OR REPLACE

Crea el procedimiento o reemplaza uno existente.

PROCEDURE nombre

Le pones un nombre.

Ejemplo:

PROCEDURE mostrar_usuario

Parámetros

Sirven para pasar información.

Hay 3 tipos.

IN

Recibe datos.

(p_id IN NUMBER)

Ejemplo:

Le pasas un id.

OUT

Devuelve datos.

(p_total OUT NUMBER)

Ejemplo:

Devuelve el total calculado.

IN OUT

Recibe y devuelve.

Menos frecuente.

IS

Aquí declaras variables.

```
IS  
  v_total NUMBER;
```

BEGIN

Empieza la lógica.

END

Termina.

Ejemplo real

```
CREATE OR REPLACE PROCEDURE contar  
(p_usuario IN NUMBER, p_total OUT NUMBER)  
IS  
BEGIN  
  SELECT COUNT(*)  
  INTO p_total  
  FROM PEDIDOS  
  WHERE ID_USUARIO = p_usuario;  
END;  
/
```

Qué hace:

1. recibe id
 2. cuenta pedidos
 3. guarda resultado
 4. devuelve resultado
-

2. CURSORES

Esto suele costar al principio.

Piensa así:

Un cursor sirve para recorrer varias filas, una por una.

Si una consulta devuelve muchas filas:

```
SELECT NOMBRE FROM PRODUCTOS;
```

No puedes meterlas todas en una variable.

Necesitas recorrerlas.

Para eso está el cursor.

Tipos de cursores

Hay 2.

A) Cursor implícito

Oracle lo crea automáticamente.

Ejemplo:

```
SELECT NOMBRE  
INTO v_nombre  
FROM PRODUCTOS  
WHERE ID_PRODUCTO = 1;
```

Oracle usa cursor interno.

No lo ves.

B) Cursor explícito

Lo creas tú.

Lo controlas manualmente.

Tiene 4 pasos.

PASO 1. Declarar

```
CURSOR c_productos IS  
  SELECT NOMBRE FROM PRODUCTOS;
```

Esto dice:

"Crea cursor llamado c_productos"

PASO 2. Abrir

```
OPEN c_productos;
```

Carga resultados.

PASO 3. Leer

```
FETCH c_productos INTO v_nombre;
```

Lee una fila.

PASO 4. Cerrar

```
CLOSE c_productos;
```

Libera memoria.

Ejemplo completo

```
DECLARE  
  CURSOR c_productos IS  
    SELECT NOMBRE FROM PRODUCTOS;  
  
  v_nombre PRODUCTOS.NOMBRE%TYPE;  
  
BEGIN  
  OPEN c_productos;  
  
  LOOP  
    FETCH c_productos INTO v_nombre;
```

```
EXIT WHEN c_productos%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_nombre);

END LOOP;

CLOSE c_productos;
END;
/
```

Atributos del cursor

Muy preguntables.

%FOUND

Encontró fila.

```
IF c%FOUND THEN
```

%NOTFOUND

No encontró fila.

Muy usado.

```
EXIT WHEN c%NOTFOUND;
```

%ROWCOUNT

Cuántas filas leyó.

```
c%ROWCOUNT
```

%ISOPEN

Dice si está abierto.

```
c%ISOPEN
```

Cursor con parámetro

Es un cursor que recibe datos.

Ejemplo:

```
CURSOR c_estado(p_estado VARCHAR2) IS
  SELECT *
  FROM PEDIDOS
  WHERE ESTADO = p_estado;
```

Uso:

```
OPEN c_estado('PENDIENTE');
```

3. TRIGGERS

Un trigger es código automático.

Se ejecuta cuando pasa algo en una tabla.

Como una alarma.

Ejemplo:

Si insertas usuario, se ejecuta solo.

Sintaxis

```
CREATE OR REPLACE TRIGGER nombre
  BEFORE/AFTER
  INSERT/UPDATE/DELETE
  ON tabla
  FOR EACH ROW
  BEGIN
    instrucciones
  END;
/
```

BEFORE y AFTER

BEFORE

Se ejecuta antes.

Ejemplo:

Validar datos antes de guardar.

AFTER

Se ejecuta después.

Ejemplo:

Guardar log.

Tipos según acción

INSERT

Cuando insertas.

UPDATE

Cuando actualizas.

DELETE

Cuando borras.

FOR EACH ROW

Hace que se ejecute por cada fila.

Sin esto, se ejecuta una sola vez.

:NEW y :OLD

Muy importante.

:NEW

Nuevo valor.

Ejemplo:

:NEW.NOMBRE

:OLD

Valor anterior.

Ejemplo:

:OLD.PRECIO

Ejemplo:

IF :NEW.PRECIO < :OLD.PRECIO THEN

Comprueba si bajó.

Ejemplo trigger

```
CREATE OR REPLACE TRIGGER trg_precio
BEFORE UPDATE ON PRODUCTOS
FOR EACH ROW
BEGIN
  IF :NEW.PRECIO < 0 THEN
    RAISE_APPLICATION_ERROR(-20001,'Error');
  END IF;
END;
/
```

Qué hace:

Antes de actualizar, revisa precio.

Si es negativo, error.

4. REGISTROS

Un registro agrupa varios campos.

Como una mini tabla en memoria.

%ROWTYPE

Copia estructura de tabla.

```
r_usuario USUARIOS%ROWTYPE;
```

Tiene todas las columnas.

Acceso:

```
r_usuario.NOMBRE
```

TYPE RECORD

Registro personalizado.

```
TYPE t_producto IS RECORD(  
  nombre VARCHAR2(50),  
  precio NUMBER  
);
```

Luego:

```
r_prod t_producto;
```

5. EXCEPCIONES

Sirven para controlar errores.

Ejemplo:

Buscar algo que no existe.

Estructura

```
EXCEPTION  
  WHEN error THEN
```

NO_DATA_FOUND

No encontró nada.

```
WHEN NO_DATA_FOUND THEN
```

TOO_MANY_ROWS

Encontró demasiadas filas.

WHEN OTHERS

Cualquier otro error.

```
WHEN OTHERS THEN
```

6. RAISE_APPLICATION_ERROR

Crea error personalizado.

```
RAISE_APPLICATION_ERROR(-20001,'Mensaje');
```

Se usa mucho en triggers.

Lo que debes saber sí o sí

Si te preguntan qué es cada cosa:

Procedimiento

Bloque reutilizable.

Cursor

Recorre filas una a una.

Trigger

Se ejecuta automáticamente.

Registro

Guarda varios datos juntos.

Excepción

Controla errores.

Para tu examen, céntrate en memorizar estas estructuras:

Procedimiento

```
CREATE OR REPLACE PROCEDURE  
IS  
BEGIN  
END;  
/
```

Cursor

```
DECLARE  
CURSOR  
OPEN  
FETCH  
EXIT WHEN  
CLOSE
```

Trigger

```
CREATE OR REPLACE TRIGGER  
BEFORE/AFTER  
FOR EACH ROW  
:NEW  
:OLD
```